

問題&心得:

- (1) 在這次實驗遇到的問題微算機開發板在LED亮燈一樣，會有兩顆燈依舊無法正常亮起，詢問過助教，可能還是開發板的問題，在SIMLAB8051的模擬輸出亮燈是正常八顆燈都可以亮的。
- (2) 而在亮燈方面，我們一開始寫的code寫成陰極會亮的版本了，結果01更改後(改成陽極版本後)還是只有打開陰極模擬出是原先正確的結果，經果一些排查後，才得知是我們如果要正常亮燈是腳位輸出為p0埠(+共陽極)，我們寫成p1埠了(忘記是p0)。
- (3) 那在這次實驗(以及上次實驗)後，我對於亮LED燈的程式(組合語言)和操作更加的熟悉了，相較於上次的LAB01之BONUS02來說，我覺得這次的LAB相較之下更加的容易上手(也可能是因為我和組員的數字加總後經過運算所要做的LAB02之BONUS01不需要做過多的更改(只需要改延遲亮燈的時間而已))，所以相對簡單。

本次實驗內容結果:

1. lab01基本:

(1) 作法:

- MOV(資料搬移)、RL(左旋轉)、RR(右旋轉)、JZ(若為0則跳躍)、DJNZ(遞減後若不為0則跳躍)、LJMP(長跳躍)、LCALL(呼叫副程式)、RET(副程式返回)。
- 程式寫法:
 - a. 程式起始:先用ORG 0H設定程式的起始位址，並用MOV P0, #0FFH初始化LED為全暗(共陽極)。
 - b. 暫存器設定:用MOV PSW, #00H設定初始的狀態，確保程式進入正確的工作區域。
 - c. 等待開始運作(按下按鍵):利用MOV A, P1不斷偵測(按鍵)狀態，如果讀到0則持續等待，直到按鍵被按下(JZ WAIT_KEY)才開始執行主程式。
 - d. LED初始化:按下按鍵後，將暫存器A設為11111110B，表示最右邊一顆LED亮起，作為跑馬燈的起始狀態。
 - e. 主迴圈控制:
 - 使用MOV P0, A將暫存器A的值輸出到P0埠，控制哪一顆LED亮。
 - 呼叫LCALL DELAY進行延遲，用巢狀迴圈控制亮燈間隔。
 - 使用RL A左旋轉，使亮點依序往左移動(若需改成由左往右，則改為用RR A)。
 - 用LJMP LOOP形成無限迴圈，讓LED連續地依順序亮起來。
 - f. 結尾:延遲副程式，用三層迴圈(R0、R1、R2)去組成延遲函式，搭配DJNZ指令去控制執行次數，最後用RET返回主程式。

(2) 程式&結果照片:

- 結果:LED會在按下按鈕後依序從右到左亮起，形成要求的跑馬燈效果，且持續地循環顯示。

```

MAIN:
MOV P0, #0FFH    ; 共陽:全暗 (1111 1111)
MOV PSW, #00H

; 等待按鈕按下
WAIT_KEY:
MOV A, P1
JZ WAIT_KEY

MOV A, #11111110B ; 第一顆LED亮 (右邊第

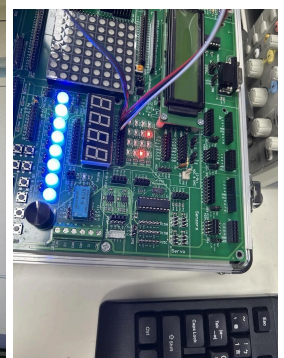
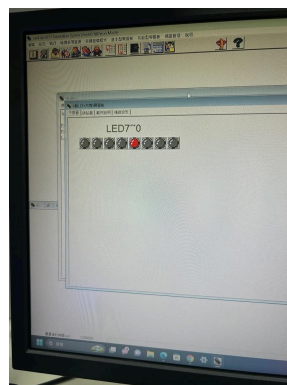
; 主迴圈
LOOP:
MOV P0, A        ; 輸出到 LED，只有一顆亮
LCALL DELAY      ; 延遲

RL A             ; 左旋轉，下一顆亮 (跑馬燈方向
; 如果要往右跑，改成 RR A

LJMP LOOP

; 延遲子程式
DELAY:
MOV R0, #10
D1: MOV R1, #40
D2: MOV R2, #249
D3: DJNZ R2, D3
    DJNZ R1, D2
        DJNZ R0, D1

```



2. bonus_1:

(1) 作法:

- 計算規則:以學號 62+43=105, 取餘數 mod 100=5, 因此 X=0、Y=5, 再根據公式: $(X+1)/2=0.5$ 秒一格、 $Z=Y \bmod 5=0$, 代表每次亮 $Z+1=1$ 顆。
- 程式寫法(和lab02基本版不同的地方):
 - a. 將原本Delay子程式中的程式迴圈次數調整成大約0.5秒, 讓LED跑馬燈移動速度加快。

(2) 程式&結果照片:

- 結果:原本Delay子程式中的迴圈次數調整成大約0.5秒, 讓LED跑馬燈移動速度更快。

```
MAIN:
MOV P0, #0FFH ; 共陽:全暗 (1111 1111)
MOV PSW, #00H

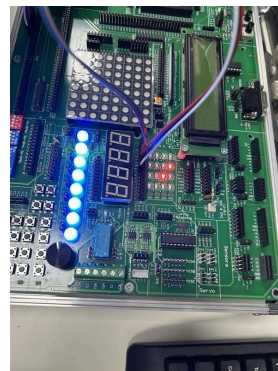
; 等待按鈕按下
WAIT_KEY:
MOV A, P1
JZ WAIT_KEY

MOV A, #1111110B ; 第一顆LED亮 (右邊第
; 主迴圈
LOOP:
MOV P0, A ; 輸出到LED, 只有一顆亮
LCALL DELAY ; 延遲

RL A ; 左旋轉, 下一顆亮 (跑馬燈方
; 如果要往右跑, 改成 RR A

LJMP LOOP

; 延遲子程式
DELAY:
MOV R0, #20
D1: MOV R1, #40
D2: MOV R2, #249
D3: DJNZ R2, D3
DJNZ R1, D2
```



3. bonus_2:

(1) 作法:

- 設計目標:呈現出Knight Rider(騎士巡邏)的效果, LED以三顆為一組來回去移動, 以模擬車頭燈的掃描動態。
- 程式寫法:
 - a. 初始設定:A=00000111B, 讓最右邊3顆LED先亮起。
 - b. 向左掃描:利用RL左移, R4作為計數器控制移動5次, 使LED由右向左掃描。
 - c. 向右掃描:利用RR右移, 同樣移動5次, 使LED從左邊再移回右邊。
 - d. 無窮迴圈:透過SJMP KNIGHT_RIDER反覆執行, 形成LED連續進行往返移動的效果。

(2) 程式&結果照片:

- 結果:執行後LED會以三顆為一組進行左右往返的掃描, 視覺效果會像影集<<Knight Rider>>車頭燈的動態光條般。

```
ORG 0000H
MAIN:
MOV P0, #0FFH ; 共陽: 初始全暗
MOV PSW, #00H

; 等待按鈕按下
WAIT_KEY:
MOV A, P1
JZ WAIT_KEY

; ===== 騎士巡邏主程式 =====
KNIGHT_RIDER:
MOV A, #11111000B ; 共陽: 左邊3顆亮 (bit7-:
MOV R4, #5 ; 移動5次
; 向右掃描 (3顆燈往右移)
SCAN_RIGHT:
MOV P0, A
LCALL DELAY_100MS

RR A ; 右旋轉
DJNZ R4, SCAN_RIGHT
; 向左掃描 (3顆燈往左移)
MOV R4, #5 ; 重設計數器
SCAN_LEFT:
MOV P0, A
LCALL DELAY_100MS
RL A ; 左旋轉
DJNZ R4, SCAN_LEFT
SJMP KNIGHT_RIDER ; 無限循環

; 100ms 延遲
DELAY_100MS:
MOV R0, #10
D1: MOV R1, #40
D2: MOV R2, #60
D3: DJNZ R2, D3
DJNZ R1, D2
DJNZ R0, D1
```

code在學校實作時要改成P0(學校是用P0)

